MATCHBAND — Andrés Alberto Flores Velasquez

Grupo: DAM 2B
Centro: I.E.S Albarregas
Módulo: Proyecto intermodular
Versión: v1.0 – 11/11/2025
Enlaces:

- Repositorio: GitHub
- Tablero: [Trello/Notion/URL]
- Figma: Proyecto

Introducción 4
1. Objetivos del proyecto 4
Objetivo general 4
Objetivos específicos 4
2. Tecnologías utilizadas 5
Frontend 5
Flutter 5
Dart 5
Backend y servicios cloud 5
Firebase Authentication 5
Cloud Firestore 5
Cloudinary 5
Herramientas de desarrollo 5
Android Studio 5
Visual Studio Code 5
Git y GitHub 6
3. Arquitectura general 6
4. Estructura del proyecto 6
Estructura principal 7
Carpeta models 7
Modelos principales 7
Carpeta screens 7
Pantallas principales 7
Carpeta services 8
Servicios principales 8
Carpeta widgets 8
Widgets reutilizables 8
Archivo main.dart 8
5. Sistema de autenticación 9
Flujo de autenticación 9
Seguridad 9
AuthGate 9
6. Base de datos Firestore 9
Colecciones principales 10
users 10
Campos principales 10
Función 10
likes 10
Campos principales 11
Función 11
passes 11
Campos principales 11
Función 11
matches 11

Campos principales..... 11
Estados utilizados..... 11
Función..... 12
proposals..... 12
Campos principales..... 12
Estados utilizados..... 12
Función..... 12
projects..... 12
Campos principales..... 12
Función..... 13
Ventajas de Firestore en MatchBand..... 13
Relación entre colecciones..... 13
7. Gestión de usuarios..... 14
Creación de perfil..... 14
Perfil musical..... 14
Información mostrada..... 14
Gestión multimedia..... 15
Flujo utilizado..... 15
Sistema de exploración de usuarios..... 15
Filtrado de perfiles..... 15
Edición de perfil..... 16
Persistencia de sesión..... 16
8. Sistema de matches y propuestas..... 16
Sistema de likes..... 16
Like..... 16
Pass..... 17
Generación de matches..... 17
Información almacenada..... 17
Estados del match..... 17
Propuestas de colaboración..... 17
Gestión de propuestas..... 18
Colaboraciones activas..... 18
Finalización de proyectos..... 18
Objetivo del sistema..... 18
9. Sistema de proyectos musicales..... 19
Publicación de proyectos..... 19
Almacenamiento multimedia..... 19
Flujo utilizado..... 19
Colección projects..... 20
Campos principales..... 20
ExploreScreen..... 20
ProjectDetailScreen..... 21
Objetivo del sistema..... 21
10. Integración con Cloudinary..... 21

Motivos de utilización 21
Funcionamiento general 22
Flujo de subida 22
Servicio cloudinary_service.dart 22
Optimización multimedia 22
Configuración aplicada 23
Seguridad 23
Ventajas obtenidas 23
Resultado final 24
11. Navegación y pantallas 24
Pantallas principales 24
LoginScreen 24
RegisterScreen 24
HomeScreen 24
RequestsScreen 25
ExploreScreen 25
ProjectDetailScreen 25
ProfileScreen 25
SelectProfileImageScreen 26
FinishProjectScreen 26
Navegación general 26
12. Seguridad y validaciones 26
Firebase Authentication 26
Validación de formularios 27
Validaciones utilizadas 27
Protección de rutas 27
Firestore Rules 27
Gestión multimedia segura 27
Restricciones utilizadas 27
13. Generación de APK 28
Proceso utilizado 28
Resultado generado 28
Ruta generada 28
Objetivo 28
14. Escalabilidad futura 28
Mejoras previstas 28
Escalabilidad técnica 29
15. Problemas encontrados y soluciones 29
Problema 1 — Gestión de imágenes 29
Solución aplicada 29
Problema 2 — Sincronización de usuarios 29
Solución aplicada 30
Problema 3 — Navegación entre pantallas 30
Solución aplicada 30

Problema 4 — Gestión multimedia web..... 30
Solución aplicada..... 30
16. Conclusión.....30

Introducción

La evolución de las tecnologías digitales ha transformado profundamente la industria musical y la manera en la que los artistas crean, comparten y desarrollan proyectos colaborativos. Sin embargo, muchos músicos independientes continúan enfrentándose a dificultades para encontrar colaboradores compatibles, gestionar proyectos musicales o mostrar su trabajo dentro de una comunidad especializada.

Actualmente, gran parte de las conexiones entre artistas se producen mediante redes sociales generalistas o aplicaciones de mensajería que no están diseñadas específicamente para cubrir las necesidades del entorno musical colaborativo. Esto provoca procesos poco organizados, dificultad para descubrir perfiles compatibles y una limitada visibilidad para músicos emergentes.

MatchBand nace como una solución multiplataforma orientada a conectar músicos mediante un sistema digital de matching y colaboración musical. La aplicación permite crear perfiles personalizados, descubrir artistas según instrumentos y géneros musicales, enviar propuestas de colaboración y publicar proyectos realizados dentro de la comunidad.

El desarrollo del proyecto se ha realizado utilizando Flutter como framework principal para el frontend multiplataforma, Firebase como infraestructura cloud para autenticación y gestión de datos en tiempo real, y Cloudinary como servicio externo para el almacenamiento multimedia.

El objetivo principal del proyecto es crear una plataforma moderna, escalable y funcional que facilite la colaboración musical entre artistas independientes mediante una experiencia intuitiva y centralizada.

1. Objetivos del proyecto

Objetivo general

Desarrollar una aplicación multiplataforma orientada a la colaboración musical entre artistas independientes, permitiendo conectar músicos mediante un sistema digital de matching, propuestas y publicación de proyectos.

Objetivos específicos

- Implementar un sistema de registro e inicio de sesión seguro.
- Permitir la creación de perfiles musicales personalizados.
- Desarrollar un sistema de descubrimiento de músicos compatible con distintos géneros e instrumentos.
- Implementar un sistema de likes y matches entre usuarios.
- Gestionar propuestas de colaboración musicales.
- Permitir la publicación de proyectos musicales finalizados.

- Integrar almacenamiento multimedia externo mediante Cloudinary.
- Diseñar una interfaz moderna e intuitiva orientada a dispositivos móviles.
- Generar una aplicación funcional multiplataforma utilizando Flutter.
- Utilizar servicios cloud para garantizar escalabilidad y sincronización en tiempo real.

2. Tecnologías utilizadas

Frontend

Flutter

Framework multiplataforma desarrollado por Google utilizado para construir toda la interfaz gráfica de la aplicación mediante un único código fuente.

Dart

Lenguaje de programación principal utilizado durante el desarrollo de la aplicación.

Backend y servicios cloud

Firebase Authentication

Servicio utilizado para la autenticación y gestión de usuarios registrados dentro de la plataforma.

Cloud Firestore

Base de datos NoSQL utilizada para almacenar información de usuarios, matches, propuestas y proyectos musicales.

Cloudinary

Servicio externo utilizado para el almacenamiento y gestión de imágenes de perfil, banners y contenido multimedia.

Herramientas de desarrollo

Android Studio

Entorno principal utilizado para el desarrollo y compilación de la aplicación Android.

Visual Studio Code

Editor utilizado para tareas de desarrollo rápido y edición de archivos.

Git y GitHub

Herramientas utilizadas para control de versiones y gestión del repositorio del proyecto.

3. Arquitectura general

La aplicación MatchBand sigue una arquitectura basada en servicios, separando la lógica de negocio, la gestión de datos y la interfaz de usuario en diferentes capas organizadas.

La arquitectura general se divide en tres bloques principales:

- Frontend desarrollado en Flutter.
- Servicios cloud mediante Firebase.
- Gestión multimedia externa mediante Cloudinary.

Flutter actúa como cliente principal de la aplicación, encargándose de la navegación, renderizado de interfaces y comunicación con los servicios externos.

Firebase proporciona autenticación de usuarios y almacenamiento de información en tiempo real mediante Cloud Firestore.

Cloudinary se utiliza como sistema externo de almacenamiento multimedia para alojar imágenes de perfil, banners y futuras portadas de proyectos musicales.

La aplicación utiliza una arquitectura modular basada en:

- Screens
- Services
- Models
- Widgets reutilizables

Esta organización facilita la escalabilidad, el mantenimiento y futuras ampliaciones del sistema.

4. Estructura del proyecto

La estructura interna del proyecto se ha organizado siguiendo una arquitectura modular para facilitar el mantenimiento, escalabilidad y reutilización del código.

La aplicación se divide en diferentes carpetas según su responsabilidad dentro del sistema.

Estructura principal

```
lib/
    │
    ├── models/
    ├── screens/
    ├── services/
    ├── widgets/
    ├── firebase_options.dart
    └── main.dart
```

Carpeta models

Contiene las clases utilizadas para representar las entidades principales de la aplicación.

Modelos principales

- `app_user.dart`
  - Representa los usuarios de la plataforma.
- `project.dart`
  - Representa los proyectos musicales publicados.
- `onboarding_data.dart`
  - Gestiona datos temporales durante el onboarding.

Carpeta screens

Contiene todas las pantallas visuales de la aplicación.

Pantallas principales

- `login_screen.dart`
- `register_screen.dart`
- `home_screen.dart`
- `profile_screen.dart`
- `explore_screen.dart`
- `requests_screen.dart`
- `proposal_screen.dart`
- `finish_project_screen.dart`

- `project_detail_screen.dart`
- `select_profile_image_screen.dart`

Cada pantalla se encarga de gestionar una funcionalidad concreta dentro de la aplicación.

Carpeta services

Contiene la lógica de negocio y comunicación con Firebase y Cloudinary.

Servicios principales

- `auth_service.dart`
  - Gestión de autenticación.
- `user_services.dart`
  - Gestión de perfiles y usuarios.
- `match_services.dart`
  - Gestión de likes, matches y propuestas.
- `project_service.dart`
  - Gestión de proyectos musicales.
- `cloudinary_service.dart`
  - Subida de imágenes a Cloudinary.

Carpeta widgets

Contiene componentes reutilizables utilizados en múltiples pantallas.

Widgets reutilizables

- `bottom_nav.dart`
- `action_button.dart`
- `tag_widget.dart`

Esta separación permite reutilizar componentes visuales y mantener una interfaz consistente.

Archivo main.dart

Es el punto de entrada principal de la aplicación.

Se encarga de:

- Inicializar Firebase.
- Configurar la aplicación Flutter.
- Cargar la navegación inicial.
- Gestionar el tema visual principal.

5. Sistema de autenticación

La autenticación de usuarios se ha implementado utilizando Firebase Authentication.

El sistema permite:

- Registro mediante correo electrónico y contraseña.
- Inicio de sesión seguro.
- Persistencia de sesión.
- Cierre de sesión.
- Gestión automática de usuarios autenticados.

Flujo de autenticación

1. El usuario crea una cuenta.
2. Firebase valida las credenciales.
3. Se genera un usuario autenticado.
4. Se almacena información adicional en Firestore.
5. El usuario accede al onboarding inicial.

Seguridad

Firebase Authentication gestiona automáticamente:

- Encriptación de contraseñas.
- Tokens de autenticación.
- Gestión de sesiones.
- Validación de usuarios.

Esto permite aumentar la seguridad del sistema sin necesidad de desarrollar un backend personalizado de autenticación.

AuthGate

La aplicación utiliza un sistema denominado `AuthGate` para comprobar automáticamente si existe una sesión activa.

Dependiendo del estado del usuario:

- Si existe sesión → acceso a la aplicación.
- Si no existe sesión → pantalla de login.

6. Base de datos Firestore

La aplicación utiliza Cloud Firestore como sistema principal de almacenamiento de datos.

Firestore es una base de datos NoSQL en tiempo real proporcionada por Firebase, orientada a aplicaciones escalables y sincronización instantánea entre dispositivos.

La estructura de la base de datos se organiza mediante colecciones y documentos.

Colecciones principales

users

Almacena la información de los usuarios registrados.

Campos principales

uid

email

artistAlias

description

contactMethod

profileImage

bannerImage

genres

instruments

profileCompleted

createdAt

updatedAt

Función

- Gestión de perfiles musicales.
- Información pública de usuarios.
- Datos utilizados en el feed principal.

likes

Almacena los likes enviados entre usuarios.

Campos principales

fromUserId

toUserId

createdAt

Función

- Detectar interés entre músicos.
- Generar matches compatibles.

passes

Almacena perfiles descartados por el usuario.

Campos principales

fromUserId

toUserId

createdAt

Función

- Evitar mostrar repetidamente perfiles descartados.

matches

Almacena matches generados entre usuarios.

Campos principales

users

status

createdAt

updatedAt

Estados utilizados

active

finished

cancelled

Función

- Gestionar colaboraciones activas.
- Controlar estado de proyectos.

proposals

Almacena propuestas de colaboración enviadas entre músicos.

Campos principales

fromUserId

toUserId

projectName

message

status

createdAt

Estados utilizados

pending

accepted

rejected

Función

- Gestionar solicitudes de colaboración.

projects

Almacena proyectos musicales publicados.

Campos principales

ownerUserId

artistAlias

title

description

genres

coverImage

spotifyUrl

youtubeUrl

votes

createdAt

Función

- Mostrar proyectos publicados en Explore.
- Crear rankings comunitarios.
- Mostrar trabajos colaborativos finalizados.

Ventajas de Firestore en MatchBand

- Sincronización en tiempo real.
- Arquitectura serverless.
- Escalabilidad automática.
- Integración directa con Flutter.
- Seguridad mediante reglas Firebase.
- Baja complejidad de mantenimiento.

Relación entre colecciones

users → likes → matches

users → proposals

users → projects

matches → projects

La relación entre colecciones permite mantener la lógica colaborativa de la plataforma sin necesidad de bases de datos relacionales tradicionales.

7. Gestión de usuarios

La gestión de usuarios constituye uno de los módulos principales de la aplicación, permitiendo la creación, modificación y visualización de perfiles musicales personalizados.

Cada usuario dispone de un perfil propio almacenado en Cloud Firestore.

Creación de perfil

Tras completar el registro e inicio de sesión, el usuario debe realizar un proceso de onboarding inicial.

Durante este proceso se recopila información relacionada con:

- Alias artístico.
- Descripción musical.
- Método de contacto.
- Instrumentos.
- Géneros musicales.

Una vez completado el onboarding, la información se almacena en la colección `users`.

Perfil musical

Cada perfil incluye información personalizada orientada a facilitar futuras colaboraciones entre músicos.

Información mostrada

- Imagen de perfil

- Banner personalizado

- Alias artístico

- Descripción

- Géneros musicales

- Instrumentos

- Método de contacto

- Actividad dentro de la plataforma

Gestión multimedia

Las imágenes de perfil y banners se almacenan utilizando Cloudinary como sistema externo de almacenamiento multimedia.

Flujo utilizado

Flutter

→ Selección de imagen

→ Subida a Cloudinary

→ Obtención de URL

→ Almacenamiento URL en Firestore

Este enfoque permite reducir carga sobre Firebase y optimizar la entrega de contenido multimedia mediante CDN.

Sistema de exploración de usuarios

La aplicación incorpora un sistema de exploración musical inspirado en plataformas de matching.

Los usuarios pueden:

- Visualizar perfiles compatibles.
- Dar like a perfiles.
- Descartar perfiles.
- Generar matches automáticos.

Filtrado de perfiles

El sistema evita mostrar:

- El propio usuario autenticado.
- Usuarios previamente descartados.
- Usuarios ya valorados.
- Perfiles incompletos.

Esto permite mantener un feed limpio y funcional.

Edición de perfil

La aplicación permite modificar:

- Alias artístico.
- Descripción.
- Método de contacto.
- Imagen de perfil.
- Banner.

Los cambios se sincronizan automáticamente con Firestore.

Persistencia de sesión

La sesión del usuario permanece activa utilizando Firebase Authentication.

El sistema detecta automáticamente si existe una sesión válida mediante `AuthGate`, evitando iniciar sesión continuamente.

8. Sistema de matches y propuestas

El sistema de matches constituye la funcionalidad principal de MatchBand, permitiendo conectar músicos interesados en colaborar entre sí.

La lógica del sistema se basa en interacciones entre usuarios mediante likes y propuestas musicales.

Sistema de likes

Dentro del feed principal, los usuarios pueden interactuar con otros perfiles musicales mediante dos acciones:

- Like

- Pass

Like

Cuando un usuario da like a otro perfil:

- Se registra la interacción en la colección `likes`.
- El sistema comprueba si existe interés mutuo.

- Si ambos usuarios se han dado like, se genera automáticamente un match.

Pass

Cuando un usuario descarta un perfil:

- Se almacena en la colección `passes`.
- El perfil deja de mostrarse en el feed.

Generación de matches

Un match se genera cuando dos usuarios se han dado like mutuamente.

Información almacenada

users

status

createdAt

updatedAt

Estados del match

active

finished

cancelled

Propuestas de colaboración

Una vez generado un match, los usuarios pueden enviar propuestas musicales.

Cada propuesta incluye:

- Nombre del proyecto

- Mensaje descriptivo

- Usuario emisor

- Usuario receptor

- Estado de la propuesta

Gestión de propuestas

Las propuestas recibidas aparecen dentro de la pantalla `RequestsScreen`.

El usuario puede:

- Revisar propuestas recibidas.
- Aceptar propuestas.
- Rechazar propuestas.
- Consultar colaboraciones activas.

Colaboraciones activas

Cuando una propuesta es aceptada:

- El match pasa a estado activo.
- La colaboración aparece dentro de “Matches abiertos”.

Desde esta sección los usuarios pueden:

- Finalizar proyecto

- Cancelar colaboración

Finalización de proyectos

Al finalizar una colaboración:

- El usuario puede publicar el proyecto musical.
- Se crea un nuevo documento dentro de la colección `projects`.
- El contenido pasa automáticamente a la sección Explore.

Objetivo del sistema

El sistema de matches y propuestas busca crear un entorno colaborativo enfocado específicamente en músicos, facilitando conexiones rápidas y organizadas entre artistas compatibles.

9. Sistema de proyectos musicales

El sistema de proyectos musicales permite publicar colaboraciones finalizadas dentro de la comunidad MatchBand.

Su objetivo es ofrecer visibilidad a los trabajos realizados por los usuarios y fomentar la interacción dentro de la plataforma.

Publicación de proyectos

Cuando una colaboración activa finaliza, los usuarios pueden publicar un proyecto musical mediante la pantalla `FinishProjectScreen`.

Durante la publicación se recopila información como:

- Título del proyecto

- Descripción

- Géneros musicales

- Imagen de portada

- Enlace Spotify

- Enlace YouTube

Almacenamiento multimedia

Las portadas de los proyectos se almacenan utilizando Cloudinary.

Flujo utilizado

Usuario

→ Selecciona imagen

→ Flutter sube imagen a Cloudinary

→ Cloudinary devuelve URL

→ URL almacenada en Firestore

Este enfoque permite optimizar el rendimiento y evitar almacenar archivos binarios directamente en Firebase.

Colección projects

Los proyectos publicados se almacenan en la colección `projects` de Firestore.

Campos principales

ownerUserId

artistAlias

title

description

genres

coverImage

spotifyUrl

youtubeUrl

votes

createdAt

ExploreScreen

La pantalla `ExploreScreen` actúa como comunidad principal de proyectos publicados.

El sistema muestra:

- Proyecto destacado

- Ranking de proyectos

- Información del artista

- Géneros musicales

- Número de votos

Los proyectos se ordenan automáticamente según popularidad y votos recibidos.

ProjectDetailScreen

Cada proyecto dispone de una pantalla de detalle individual.

Esta pantalla muestra:

- Portada

- Título

- Artista

- Descripción completa

- Géneros

- Votos

- Enlaces externos

Objetivo del sistema

El sistema de proyectos musicales busca convertir MatchBand en una comunidad creativa donde los músicos no solo puedan conocerse, sino también mostrar públicamente el resultado de sus colaboraciones.

10. Integración con Cloudinary

La aplicación integra Cloudinary como plataforma externa para la gestión y almacenamiento multimedia.

Cloudinary se utiliza para almacenar:

- Imágenes de perfil

- Banners de usuario

- Portadas de proyectos

Motivos de utilización

Inicialmente se valoró el uso de Firebase Storage, sin embargo se decidió utilizar Cloudinary debido a:

- Facilidad de integración.
- CDN incorporado.
- Optimización automática de imágenes.

- Gestión multimedia especializada.
- Reducción de carga sobre Firebase.
- URLs públicas optimizadas.

Funcionamiento general

La subida de imágenes se realiza directamente desde Flutter mediante peticiones HTTP al servicio Cloudinary.

Flujo de subida

Usuario selecciona imagen

→ Flutter procesa imagen

→ Imagen enviada a Cloudinary

→ Cloudinary devuelve URL segura

→ URL almacenada en Firestore

Servicio cloudinary_service.dart

La aplicación utiliza un servicio dedicado denominado:

cloudinary_service.dart

Este servicio se encarga de:

- Gestionar peticiones HTTP.
- Enviar imágenes mediante multipart/form-data.
- Obtener URLs seguras.
- Gestionar errores de subida.

Optimización multimedia

Durante la subida se aplican restricciones y optimizaciones para mejorar rendimiento y experiencia de usuario.

Configuración aplicada

Formatos permitidos:

- JPG

- JPEG

- PNG

- WEBP

Compresión automática:

- imageQuality: 80

Resolución máxima:

- Perfil: 1000px

- Banner: 1600px

Seguridad

La aplicación utiliza un Upload Preset unsigned configurado en Cloudinary.

Esto permite:

- Subidas directas desde la aplicación.
- Simplificación del backend.
- Integración rápida durante el desarrollo.

Ventajas obtenidas

La integración de Cloudinary aporta:

- Mejor rendimiento multimedia.
- Carga rápida de imágenes.
- Escalabilidad.
- Menor complejidad backend.
- Entrega optimizada mediante CDN global.

Resultado final

Gracias a Cloudinary, MatchBand puede gestionar contenido multimedia dinámico de manera eficiente sin depender exclusivamente de Firebase Storage.

11. Navegación y pantallas

La aplicación MatchBand utiliza un sistema de navegación basado en pantallas independientes desarrolladas con Flutter.

La navegación principal se realiza mediante una barra inferior (`BottomNav`) compartida entre las secciones más importantes de la aplicación.

Pantallas principales

LoginScreen

Permite al usuario iniciar sesión mediante correo electrónico y contraseña utilizando Firebase Authentication.

RegisterScreen

Permite crear nuevas cuentas de usuario.

Durante el registro:

- Se validan credenciales.
- Se crea el usuario en Firebase Authentication.
- Se prepara el perfil inicial.

HomeScreen

Pantalla principal de exploración de músicos.

Incluye:

- Feed de perfiles.
- Sistema de likes.
- Sistema de descartes.
- Generación de matches.

La interfaz está inspirada en aplicaciones de matching modernas.

RequestsScreen

Pantalla dedicada a:

- Propuestas recibidas.
- Colaboraciones activas.
- Gestión de matches.

Permite aceptar, cancelar o finalizar colaboraciones.

ExploreScreen

Pantalla comunitaria donde se muestran proyectos publicados.

Incluye:

- Proyecto destacado.
- Ranking de proyectos.
- Acceso a detalles individuales.

ProjectDetailScreen

Muestra información detallada de un proyecto musical publicado.

Incluye:

- Portada.
- Descripción.
- Géneros.
- Enlaces externos.
- Información del artista.

ProfileScreen

Pantalla de perfil personal del usuario.

Permite:

- Visualizar información personal.
- Editar perfil.
- Cambiar imágenes.
- Consultar actividad.

SelectProfileImageScreen

Pantalla destinada a la subida y selección de imágenes mediante Cloudinary.

FinishProjectScreen

Pantalla utilizada para publicar proyectos musicales finalizados.

Navegación general

El flujo principal de navegación es:

Login/Register

→ Onboarding

→ Home

→ Match

→ Proposal

→ Collaboration

→ Publish Project

→ Explore

12. Seguridad y validaciones

La aplicación incorpora diferentes mecanismos de validación y seguridad para garantizar un funcionamiento estable y proteger el acceso a los datos de usuario.

Firebase Authentication

La autenticación se realiza mediante Firebase Authentication, permitiendo:

- Gestión segura de usuarios.
- Persistencia de sesión.
- Encriptación automática de contraseñas.
- Control de acceso autenticado.

Validación de formularios

Los formularios implementan validaciones básicas para evitar datos inválidos.

Validaciones utilizadas

- Campos obligatorios

- Contraseñas mínimas

- Correos válidos

- Prevención de campos vacíos

Protección de rutas

La aplicación utiliza `AuthGate` para comprobar automáticamente si existe una sesión válida antes de permitir acceso a la aplicación.

Firestore Rules

La base de datos se protege mediante reglas de seguridad de Firebase.

Estas reglas permiten:

- Restringir acceso a usuarios autenticados.
- Evitar modificaciones no autorizadas.
- Limitar escritura según propietario del documento.

Gestión multimedia segura

Cloudinary utiliza Upload Presets configurados específicamente para permitir únicamente subida de imágenes compatibles.

Restricciones utilizadas

- JPG

- JPEG

- PNG

- WEBP

Además:

- Se limita calidad de imagen.
- Se controla tamaño máximo recomendado.
- Se optimiza resolución automáticamente.

13. Generación de APK

La aplicación ha sido compilada en formato APK para dispositivos Android utilizando Flutter.

Proceso utilizado

flutter build apk --release

Resultado generado

La compilación genera una versión optimizada lista para instalación en dispositivos Android.

Ruta generada

build/app/outputs/flutter-apk/app-release.apk

Objetivo

La generación de APK permite:

- Distribución de la aplicación.
- Pruebas reales en dispositivos móviles.
- Validación funcional completa.
- Presentación del proyecto final.

14. Escalabilidad futura

La arquitectura desarrollada permite ampliar funcionalidades futuras sin modificar la base principal del sistema.

Mejoras previstas

- Chat en tiempo real

- Sistema de comentarios

- Sistema de votos avanzado

- Integración Spotify API

- Integración YouTube API

- Notificaciones push

- Recomendaciones musicales

- Seguimiento de artistas

- Verificación de perfiles

- Moderación de contenido

Escalabilidad técnica

Gracias al uso de Firebase y Cloudinary, la aplicación puede escalar fácilmente sin necesidad de infraestructura propia compleja.

---

15. Problemas encontrados y soluciones

Durante el desarrollo del proyecto surgieron diferentes problemas técnicos relacionados con integración cloud, sincronización de datos y estructura de navegación.

Problema 1 — Gestión de imágenes

Inicialmente se valoró utilizar Firebase Storage para almacenar imágenes.

Solución aplicada

Se decidió utilizar Cloudinary debido a:

- Mejor optimización multimedia.
- Integración más sencilla.
- CDN incorporado.
- Menor complejidad de configuración.

Problema 2 — Sincronización de usuarios

Durante las primeras pruebas, algunos usuarios no aparecían correctamente en el feed principal.

Solución aplicada

Se revisó la validación de:

- `profileCompleted`
- IDs de usuario
- Likes y passes registrados

Problema 3 — Navegación entre pantallas

Algunas pantallas no actualizaban correctamente la información tras finalizar colaboraciones.

Solución aplicada

Se implementaron:

- Recargas automáticas
- RefreshIndicator
- Navegación mediante retorno de estados

Problema 4 — Gestión multimedia web

Durante pruebas en navegador surgieron problemas relacionados con subida de imágenes mediante `image_picker`.

Solución aplicada

Se priorizó ejecución y validación en Android nativo mediante APK release.

---

16. Conclusión

MatchBand ha permitido desarrollar una plataforma multiplataforma funcional orientada a la colaboración musical entre artistas independientes.

El proyecto integra tecnologías modernas como Flutter, Firebase y Cloudinary para crear una experiencia dinámica, escalable y orientada a dispositivos móviles.

A lo largo del desarrollo se han implementado sistemas de autenticación, matching, gestión de colaboraciones y publicación de proyectos musicales, obteniendo una aplicación funcional y preparada para futuras ampliaciones.

El resultado final demuestra la viabilidad de utilizar arquitecturas cloud y desarrollo multiplataforma para construir aplicaciones modernas con una experiencia cercana a productos reales del mercado.